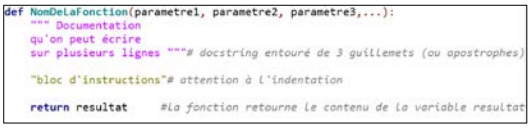
NSI S5 : Les fonctions



Dans la séquence précédente, vous avez déjà vu comment programmer une fonction :

```
def NomDeLaFonction(parametre1, parametre2, parametre3,...):
    """ Documentation
    qu'on peut écrire
    sur plusieurs lignes """# docstring entouré de 3 guillemets (ou apostrophes)

    "bloc d'instructions"# attention à l'indentation

    return resultat     #la fonction retourne le contenu de la variable resultat
```

Le but de cette séquence est d'utiliser des fonctions afin de programmer :

- Des fonctions mathématiques,
- Générateur de mots de passe
- Une grille de l'euro million,
- Tracer un colimaçon,
- Une courbe de Koch,
- Un arbre binaire.

1. Les fonctions mathématiques :

I) Ecrire une fonction qui retourne f(x) = $27x^3-27x^2-18x+8$ et affiche La valeur de f(???) = ??.
Sauver sous exo1.py et tester afficher f(0),f(0.5),f(0.25),f(0.375), f(1) et f(2)

Nous allons, afficher cette courbe pour cela, il faudra utiliser la librairie matplotlib en mettant en début de programme import matplotlib.pyplot as plt.
On définit deux listes X et Y qui vont contenir les valeurs de X et Y= f(X) en codant **X=[]** et **Y=[]**
Pour ajouter un élément dans la liste on fera :
X.append(x) et **x= -1** puis on l'incrémentera de 0.1 jusqu'à 2.
Donc on écrira :

```
while x<=2 :
        X.append(x) #on ajoute x à la liste
        x=x+0.1     #on incrémente x de 0.1
```

Rajouter les lignes de codes pour x,X et Y .
Pour tracer le graphe, on codera : **plt.plot(X,Y,) #Placement des points sur la graphique** puis : **plt.show() #Affichage du graphe**
Ajouter le graphique et son affichage puis sauver sous exo1.py et tester.

II) Ecrire une fonction qui retourne la factorielle (se représente par un !) d'un nombre entier N.
On rappelle que : N ! = 1×2×...×N-1×N.
Afficher 2!; 10 !; 20 !; 50 !
Comparer les résultats avec factorial() du module math à importer auparavant.
Sauver sous exo2.py.

III) A la place d'afficher des valeurs définies dans le programme, modifier le afin qu'on demande à l'utilisateur la valeur de N dont il veut la factorielle. Tester avec 0 !,10 !-10 !
Sauver sous exo3.py

Nous allons maintenant gérer les erreurs si l'utilisateur entre un nombre négatif (la factorielle n'existe pas).
Pour cela, il faut utiliser une assertion.
On va rajouter dans la fonction les deux lignes suivantes pour tester :

```
try :
        assert N>=0
        resultat=1
        for i in range (0,N):
                resultat=resultat*(i+1)
```

puis ce qu'il faut faire si la condition est fausse :

```
except AssertionError :
        print("On ne peut pas calculer la factorielle d'un nombre négatif")
        resultat="n'existe pas"
return resultat
```

IV) Rajouter cette assertion, tester 0 !,10 !-10 ! et sauver sous exo4.py.

```
def NomDeLaFonction(parametre1, parametre2, parametre3,...):
    """ Documentation
    qu'on peut écrire
    sur plusieurs lignes """# docstring entouré de 3 guillemets (ou apostrophes)

    "bloc d'instructions"# attention à l'indentation

    return resultat     #la fonction retourne le contenu de la variable resultat
```

2. Générateur de mot de passe :

V) A l'aide de la fonction randint() du module random, écrire une fonction qui retourne un mot de passe de longueur N (chiffres, lettres minuscules ou majuscules).
On donne :
chaine = '0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz'
Sauver sous exo5.py et afficher un mot_de_passe(6) et un mot_de_passe(10).

3. Grille de l'Euro Millions :

VI) Ecrire une fonction qui retourne une grille de numéros du jeu Euro Millions.
Il faut 5 numéros entre 1 et 50 et 2 numéros chance entre 1 et 11.
On utilisera la fonction sample() du module random à importer au début du programme en codant (sample(tabVal,nbVal)). Il va choisir nbValeurs dans la tabVal.
Exemple : tabVal=[0,1,2,3,4,5] et nbVal=3
alors il choisira 3 valeurs au hasard dans tabVal par exemple [1,4,2].
Sauver sous exo6.py et tester

4. Récursivité

a) La factorielle

Revenons sur notre exercice de la factorielle.
On pourrait aussi dire que factorielle(n) =
Si n=1 ou 0 cela vaut 1
Sinon cela vaut n*factorielle(n-1)
vérifions avec n=2 : 2 !=2*1 !=2
avec n=3 :3 !=3*2 !=3*2=6…….
Lorsqu'une fonction s'appelle elle-même elle est dite récursive.

VII) Programme la factorielle de façon récursive
Afficher 2!; 10 !; 20 !; 50 !
Comparer les résultats avec factorial() du module math à importer auparavant.
Sauver sous exo7.py

Passons à des choses un peu plus graphiques. Pour cette partie, nous allons utiliser le module turtle qui permet de tracer des lignes ou des cercles.
Il faudra toujours commencer par importer la librairie : from turtle import *
Les principales fonctions de ce module sont :

Fonction	Description
reset()	On efface tout et on recommence
goto(x,y)	Aller à l'endroit de coordonnées x et y
forward(distance)	Avancer d'une distance donnée
backward(distance)	Reculer
up()	Relever le crayon (pour pouvoir avancer sans dessiner)
down()	Abaisser le crayon (pour pouvoir recommencer à dessiner)
color(couleur)	Couleur peut être une chaîne prédéfinie ('red', 'blue', 'green', etc.)
left(angle)	Tourner à gauche d'un angle donné (exprimé en degré)
right(angle)	Tourner à droite
width(épaisseur)	Choisir l'épaisseur du tracé
write(texte)	texte doit être une chaîne de caractères délimitée avec des " ou des '
setup(coordonnées)	Ex : setup(width=300,height=300,startx=0,starty=0)
exitonclick()	Ferme la fenêtre turtle lorsqu'on clique
done()	Sort du module turtle
speed(x)	Vitesse du tracé si x vaut 0 c'est le plus rapide

```
def NomDeLaFonction(parametre1, parametre2, parametre3,...):
    """ Documentation
    qu'on peut écrire
    sur plusieurs lignes """# docstring entouré de 3 guillemets (ou apostrophes)

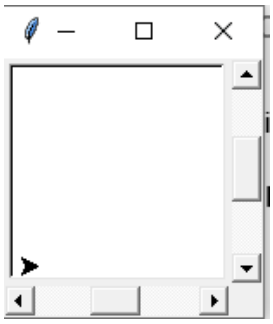
    "bloc d'instructions"# attention à l'indentation

    return resultat     #la fonction retourne le contenu de la variable resultat
```

b) Le colimaçon

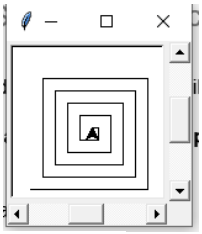
Commençons par importer le module puis définir la taille de la fenêtre :

```
from turtle import *
setup(150,150) #dimension de la fenêtre(cela comprend la zone de dessin et les bords de la fernêtre)
```



Plaçons ensuite le stylo noir en bas à gauche :

```
color('black')
up() # lève le stylo
backward(50) # recule de 25 pixels
right(90) # tourne de 90 degrés vers la droite
forward(50) # avance de 50 pixels
left(90) # tourne de 90 degrés vers la gauche
down() # pose le stylo
exitonclick() #Ferme la fenêtre au clic
```



Si vous exécuter le programme, le stylo se trouve en bas à gauche et pointe vers la droite.

Maintenant nous allons programmer par récursivité un colimaçon d'angle 90 ° donc l'arrête diminue de 5 pixels à chaque rotation.

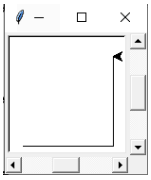


Pour n=0, on trace un trait de longueur puis on tourne de 90°.

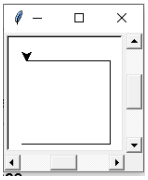
Sinon :

- on avance de (longueur-5)
- puis on tourne de 90°
- puis on fait on rappelle par récursivité la fonction colimacon

n=0



n=1



n=2

```
def colimacon(n,longueur):
    if n==0:
        forward(longueur) # avance
        left(angle)#tourne vers la gauche de angle

    else:
        forward(longueur-5) # avance
        left(angle) #tourne vers la gauche de angle
        colimacon(n-1,longueur-5)

colimacon(20,100) # exécute la fonction
exitonclick() #Ferme la fenêtre au clic
```

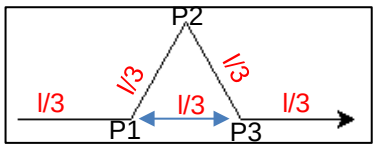
VIII) Programme le colimaçon de façon récursive.
Sauver sous exo8.py et tester avec n=20 et longueur=100.

c) Courbe de Koch

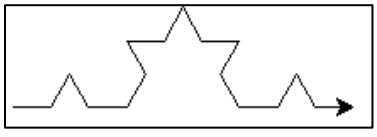
Cette courbe consiste à couper par récursivité un segment en 3 partie égales et à remplacer celle du milieu par deux segments de même dimensions et d'angle 60°.



n=0



n=1



n=2

Une courbe de Koch va être composée d'une courbe de Koch d'ordre plus petit.
Si n=0 alors la courbe est un segment de longueur l
Sinon c'est composé de 4 courbes d'ordre n-1 de longueur l/3.

Dans un premier temps, il faut définir les angles de rotation entre chaque segment si n>0.

```
def NomDeLaFonction(parametre1, parametre2, parametre3,...):
    """ Documentation
    qu'on peut écrire
    sur plusieurs lignes """# docstring entouré de 3 guillemets (ou apostrophes)

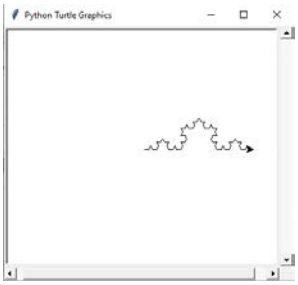
    "bloc d'instructions"# attention à l'indentation

    return resultat     #la fonction retourne le contenu de la variable resultat
```

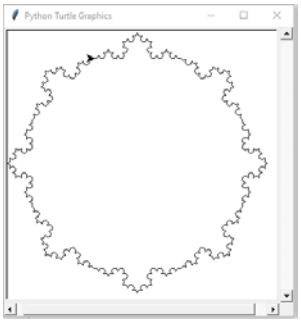
IX) Compléter le tableau ci-dessous.

Point	Sens	Angle
P1	left	60
P2	right	
P3	left	

Ecrire la fonction def koch(n,l).
Créer le programme qui trace une courbe de Koch (n,l) dont la fenêtre mesure 400,400 pixels², on mettra la vitesse à 0(speed(0)) et on oubliera pas de rajouter l'instruction exitonclick() pour pouvoir sortir proprement de turtle.
Sauver sous exo9.py et tester avec n=3 et longueur=200.

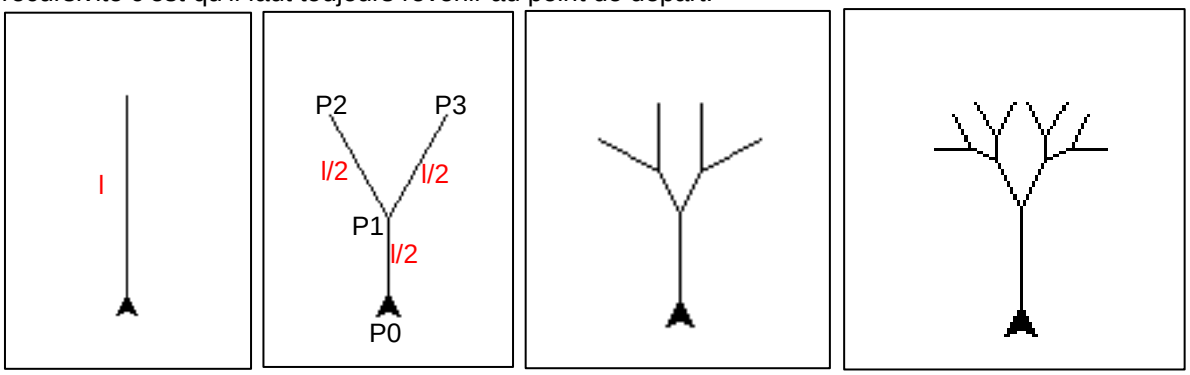


Modifier le programme afin de vous déplacer au point (-70, 150) puis appeler 8 fois la courbe de Koch avec n=3, l=120 et vous ferez une rotation right de 45° entre chacune.
Sauver sous exo9.py et tester (vous devez obtenir l'image ci-contre).



d) Arbre binaire

Un arbre binaire est composé d'un tronc principal de longueur l puis de branche d'ordre n-1 et de longueur l/2. La particularité de cette récursivité c'est qu'il faut toujours revenir au point de départ.

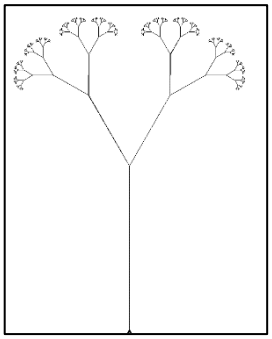


n=0 n=1 n=2 n=3

Dans un premier temps, il faut définir les angles de rotation entre chaque segment si n>0.

X) Compléter le tableau ci-dessous.

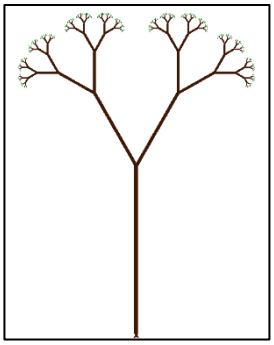
Points	Sens	Angle
P1vers P2	left	30
P1 vers P3	right	
P1 vers P0	left	



Ecrire la fonction def arbre(n,longueur).
Pour n=0, on fera un forward de longueur l puis un backward de cette même longueur.
Pour n>0 :

- on avancera de longueur/2
- puis on effectuera la première rotation
- on appliquera la fonction (arbre(n-1,longueur/2))
- puis on effectuera la deuxième rotation
- on appliquera la fonction (arbre(n-1,longueur/2))
- puis on effectuera la troisième rotation
- puis un backward de longueur/2

Créer le programme qui trace l'arbre (n,l) dont la fenêtre mesure 800,800 pixels², on mettra la vitesse à 0(speed(0)), on se placera en bas au milieu en levant le stylo puis en appliquant les 3 fonctions suivantes **right(90) # tourne de 90 degrés vers la droite**
forward(300) # avance de 300 pixels
left(180) # fait un demi-tour
et on oubliera pas de rajouter l'instruction exitonclick() pour pouvoir sortir proprement de turtle.
Sauver sous exo10.py et tester avec n=8 et longueur=700.



Pour ceux qui aime cela, modifier le programme pour obtenir l'arbre ci-contre et sauver sous exo10bis.py.